Module 02

Python asyncio

Concurrency without threads — event loops, coroutines, and async patterns

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. Why Do We Need Concurrency?

Imagine your program fetches data from 10 websites. If done one at a time it might take 10 × 2 seconds = 20 seconds. If you could do all 10 at once, it would take ~2 seconds. There are three main ways to achieve this in Python:

Approach	How it works	Best for
Threads	OS runs multiple threads in parallel	I/O-bound (many blocking calls)
Processes	OS runs separate Python processes	CPU-bound (heavy computation)
asyncio	One thread, cooperative switching	I/O-bound, very many tasks

2. Coroutines — The Async Function

A **coroutine** is a function defined with async def. Calling it does NOT run it immediately — it returns a coroutine object. You must await it to actually execute it. The await keyword tells Python: 'pause here and let other coroutines run while we wait for this I/O.'

```
import asyncio

# Regular function — runs synchronously (blocks)
def slow_sync():
    import time
    time.sleep(2)
    return "done"

# Async function — can be paused, lets others run during the wait
async def slow_async():
    await asyncio.sleep(2)    # yields control to the event loop
    return "done"

# Running a coroutine
async def main():
    result = await slow_async()   # awaiting it actually runs it
    print(result)

asyncio.run(main())   # this is the entry point — starts the event loop
```

> ***Note:*** *asyncio.sleep() is non-blocking. time.sleep() is blocking — it freezes the whole program. NEVER use time.sleep() inside async code.*

3. asyncio.gather — Run Multiple Tasks at Once

The most powerful asyncio primitive. It runs several coroutines **concurrently** and waits for all of them to finish. This is how you fetch 10 websites at the same time.

```
import asyncio

async def fetch(site: str) -> str:
    await asyncio.sleep(1)    # simulating a 1-second network call
    return f"data from {site}"

async def main():
```

```
    # Sequential — takes 3 seconds
    r1 = await fetch("site1.com")
    r2 = await fetch("site2.com")
    r3 = await fetch("site3.com")

    # Concurrent with gather — takes ~1 second
    r1, r2, r3 = await asyncio.gather(
        fetch("site1.com"),
        fetch("site2.com"),
        fetch("site3.com"),
    )
    print(r1, r2, r3)

asyncio.run(main())
```

You can also use return_exceptions=True to prevent one failure from cancelling others — individual results can be Exception objects, which you check with isinstance(result, Exception).

```
# gather with error handling
results = await asyncio.gather(
    fetch("good-site.com"),
    fetch("broken-site.com"),   # might raise
    return_exceptions=True,
)
for r in results:
    if isinstance(r, Exception):
        print(f"Failed: {r}")
    else:
        print(f"Got: {r}")
```

4. asyncio.Semaphore — Limit Concurrency

Running 1000 tasks at once might crash your server or get you rate-limited. A **Semaphore** is a counter that limits how many tasks can run simultaneously.

```
import asyncio

sem = asyncio.Semaphore(3)   # max 3 concurrent tasks

async def fetch(url: str) -> str:
    async with sem:          # waits if 3 tasks are already running
        await asyncio.sleep(1)
        return f"data from {url}"

async def main():
    urls = [f"site{i}.com" for i in range(10)]
    results = await asyncio.gather(*[fetch(u) for u in urls])
    # Even with 10 tasks, only 3 run at a time

asyncio.run(main())
```

> ***Note:*** *In ContentAutomation2 the research agent uses Semaphore(3) to cap concurrent browser instances and Semaphore(2) to limit parallel Supabase batch-dedup calls.*

5. asyncio.Lock — Protect Shared State

When multiple coroutines share a variable (like a counter), reads and writes can interleave in unexpected ways. A **Lock** ensures only one coroutine touches the shared variable at a time.

```
import asyncio

lock = asyncio.Lock()
counter = 0

async def increment():
    global counter
    async with lock:
        # Only one coroutine is inside this block at a time
        current = counter
        await asyncio.sleep(0)   # simulate a tiny pause
        counter = current + 1

async def main():
    await asyncio.gather(*[increment() for _ in range(100)])
    print(counter)   # 100 — no race condition

asyncio.run(main())
```

6. asyncio.to_thread — Run Sync Code Without Blocking

Sometimes you must call synchronous code (like a Supabase client that only has a sync API) from async code. Wrapping it in asyncio.to_thread() runs it in a separate thread so it doesn't block the event loop.

```
import asyncio

def slow_db_call(query: str) -> list:
    # Pretend this is a synchronous DB call (blocking)
    import time
    time.sleep(0.5)
    return [{"id": 1}, {"id": 2}]

async def main():
    # WRONG — blocks the event loop for 0.5 seconds:
    # data = slow_db_call("SELECT * FROM items")

    # CORRECT — runs in a thread, event loop stays free:
    data = await asyncio.to_thread(slow_db_call, "SELECT * FROM items")
    print(data)

asyncio.run(main())
```

> ***Note:*** *ContentAutomation2 wraps all Supabase calls with asyncio.to_thread() because the supabase-py client is synchronous.*

7. asyncio.wait_for — Timeouts

```
import asyncio
```

```
async def slow_operation():
    await asyncio.sleep(10)   # takes 10 seconds
    return "result"

async def main():
    try:
        result = await asyncio.wait_for(slow_operation(), timeout=3.0)
    except asyncio.TimeoutError:
        print("Operation timed out after 3 seconds!")

asyncio.run(main())
```

8. nonlocal — Sharing State Inside Closures

When a nested function (like a task handler) needs to modify a variable in the outer function, use the nonlocal keyword.

```
async def run_pipeline():
    success_count = 0
    failure_count = 0

    async def process_item(item):
        nonlocal success_count, failure_count   # reference outer variables
        try:
            await do_work(item)
            success_count += 1
        except Exception:
            failure_count += 1

    await asyncio.gather(*[process_item(i) for i in range(10)])
    print(f"Success: {success_count}, Failures: {failure_count}")
```

> ***Tip:*** *If multiple tasks run concurrently, protect shared counters with a Lock to avoid race conditions (see ContentAutomation2's _lock pattern in tasks.py).*

9. How ContentAutomation2 Uses asyncio

The research agent processes 7 news sites in parallel with asyncio.gather. Each site processing itself fetches multiple articles concurrently (capped by Semaphore). Supabase (sync) calls are wrapped in asyncio.to_thread. Article fetches have a 90-second hard timeout via asyncio.wait_for.

10. Practice Exercises

- Exercise 1: Write a coroutine fetch(url) that sleeps 1 second and returns the url. Run 5 of them sequentially, measure time.
- Exercise 2: Now run the same 5 with asyncio.gather — measure the time difference.
- Exercise 3: Add a Semaphore(2) so only 2 run at once.
- Exercise 4: Add a shared counter protected by a Lock. Verify the count is always correct.
- Exercise 5: Wrap time.sleep(1) with asyncio.to_thread and verify the event loop stays free.

- Exercise 6: Add asyncio.wait_for with a 0.5 second timeout and handle the TimeoutError.